

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Q1. Insurance Claim Verification System

An insurance system uses three nested loops to verify policy details, claim history, and supporting

records. Recall how triple nested loops affect complexity, explain the rate of growth, and

determine both the time complexity and space complexity for this process.

Q2. Network Access Control System

A network access system checks each user request sequentially against an access control list.

Define linear search, explain how the algorithm works, and determine both the time complexity

and space complexity as the number of requests increases.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Insurance Claim Verification System – Triple Nested Loops

Concept of Triple Nested Loops

A triple nested loop consists of three loops placed inside one another. Each loop runs completely for every iteration of the outer loop. This significantly increases the number of operations performed.

Working of the System

In an insurance claim verification system, three levels of checking are performed:

1. **Outer Loop** → Verifies policy details
2. **Middle Loop** → Checks claim history
3. **Inner Loop** → Validates supporting records

For every policy, all claim histories are checked, and for each claim, all supporting records are verified. This results in repeated execution of inner operations.

Rate of Growth

If each loop runs **n times**, then:

- Outer loop runs n times
- For each iteration, middle loop runs n times
- For each of those, inner loop runs n times

Total operations = $n \times n \times n = n^3$

Thus, the growth rate is **cubic**, meaning the execution time increases very rapidly as input size increases.

Time Complexity

Let n be the number of records.

- Best Case: $O(n^3)$
- Worst Case: $O(n^3)$
- Average Case: $O(n^3)$

Time complexity is dominated by three nested loops, so it is **$O(n^3)$** .

Space Complexity

The algorithm does not require extra memory apart from loop variables.

Space Complexity = $O(1)$ (Constant space)

Advantages

- Performs thorough and detailed verification
- Ensures accuracy by checking all combinations
- Suitable for critical systems where correctness is important

Disadvantages

- Very slow for large datasets
- High computational cost
- Not scalable for real-time systems

Real-Time Application Insight

Used in systems where accuracy is more important than speed, such as fraud detection, auditing, and detailed claim verification processes.

Detailed Performance Analysis

In triple nested loops, every element is compared with every possible combination of other elements. This results in a very high number of operations.

If:

- Policies = n
- Claims per policy = n
- Records per claim = n

Total checks = n^3

Example:

If $n = 100 \rightarrow 100^3 = 10,000,000$ operations 😱

Scalability Issue

As the number of policies and claims increases, the system becomes very slow. This makes triple nested loops unsuitable for:

- Real-time processing
- Large insurance databases

- Online claim verification systems

Optimization Techniques

To reduce complexity, systems can use:

- **Database indexing** to quickly locate records
- **Parallel processing** to handle multiple verifications
- **Machine learning models** to predict valid claims
- **Filtering before looping** to reduce unnecessary checks

Practical Insight

Modern insurance systems avoid full triple nested loops by:

- Pre-validating data
- Using optimized queries
- Applying rule-based filtering

Mathematical Representation

$$T(n) = c \times n^3$$

Growth order = Cubic growth

This means if input size doubles, time increases by 8 times.

Conclusion

Triple nested loops result in cubic time complexity, making the system slow for large datasets. While suitable for small-scale verification, it becomes inefficient when the number of policies, claims, and records increases significantly.

2. Network Access Control System – Linear Search

Definition of Linear Search

Linear search is a basic searching technique where each element in a list is examined sequentially until the desired element is found or the entire list has been traversed.

Working of the Algorithm

In a Network Access Control System, linear search is used to verify user access against an Access Control List (ACL):

1. The ACL contains a list of authorized users or permissions.
2. When a user request is received, the system starts checking from the first entry.
3. Each entry is compared with the request.
4. If a match is found, access is granted immediately.
5. If no match is found, the system proceeds to the next entry.
6. If the end of the list is reached without a match, access is denied.

Algorithm (Pseudo Code)

```
for each user in ACL:  
    if user == request:  
        return "Access Granted"  
return "Access Denied"
```

Rate of Growth

The number of comparisons increases linearly with the size of the ACL. If the number of users doubles, the time taken also approximately doubles. Hence, it shows **linear growth**.

Time Complexity

Let n be the number of entries in the ACL:

- Best Case: $O(1)$ → User found at first position
- Worst Case: $O(n)$ → User at last position or not found
- Average Case: $O(n)$

Space Complexity

Only a few variables are used for comparison.

Space Complexity = $O(1)$

Advantages

- Simple and easy to implement
- No need for sorted data
- Works well for small datasets

Disadvantages

- Inefficient for large datasets
- Requires checking each element sequentially
- Slower compared to advanced searching techniques

Real-Time Application Insight

Used in small-scale systems like basic login verification, firewall rule checking, and simple permission validation systems.

Optimization Possibilities

To improve performance, the following techniques can be used instead of linear search:

- **Binary Search** (if ACL is sorted) → $O(\log n)$
- **Hashing Techniques** → $O(1)$ average time
- **Indexing or Database Lookup** → Faster access control

These methods significantly reduce search time compared to linear search.

Practical Example in Networking

In real-world systems such as:

- Firewall rule matching
- Router access filtering
- User authentication logs

Linear search may be used for small rule sets, but advanced methods are preferred for large-scale networks.

Conclusion

Linear search is suitable for small access control systems but becomes inefficient as the number of users grows due to its linear time complexity.